Sistema de Gestión para Biblioteca Universitaria (Python)

INTRODUCCIÓN

En el contexto de la transformación digital de las instituciones de educación superior, la incorporación de herramientas informáticas en los servicios bibliotecarios representa una oportunidad para mejorar la organización de los recursos académicos, optimizar los procedimientos administrativos y fortalecer el acceso a la información. La gestión bibliotecaria requiere controlar adecuadamente los materiales bibliográficos, identificar a los usuarios y mantener un registro actualizado de los préstamos y las devoluciones. En este sentido, la investigación de Chen (2023) sobre la aplicación de la computación en la nube a los sistemas de gestión de bibliotecas universitarias evidencia la relevancia de las soluciones tecnológicas para modernizar estos servicios. Asimismo, el estudio de *A cloud-based approach to library management solution for college libraries* destaca alternativas de infraestructura digital orientadas a facilitar la prestación de servicios bibliotecarios (A cloud-based approach to library management solution for college libraries, 2021). Estas investigaciones permiten contextualizar la importancia de diseñar sistemas informáticos que respondan a las necesidades concretas de administración y control de las bibliotecas académicas.

Desde la perspectiva de la ingeniería de software, el desarrollo de una solución informática requiere identificar las necesidades de los usuarios, especificar las funcionalidades y establecer condiciones que permitan verificar el comportamiento esperado del sistema. Sommerville (2021) aborda la ingeniería de software como una disciplina que integra procesos y técnicas para desarrollar sistemas confiables y mantenibles. Bajo esta perspectiva, el diseño de un sistema de gestión para una biblioteca universitaria debe partir de requerimientos claramente definidos, como el registro de libros y estudiantes, la consulta de materiales disponibles y el control de los préstamos y las devoluciones. Estas

funcionalidades deben relacionarse mediante reglas de validación que eviten registros inconsistentes y operaciones incompatibles.

Para la implementación del sistema, el lenguaje de programación Python constituye una alternativa apropiada en el ámbito educativo, debido a que permite organizar los datos y las operaciones mediante clases y objetos. La documentación oficial de Python Software Foundation (s. f.) explica que las clases permiten agrupar datos y funcionalidades, mientras que los objetos representan instancias que mantienen atributos y comportamientos propios. En consecuencia, el enfoque de programación orientada a objetos posibilita representar los principales componentes del sistema mediante clases como Libro, Estudiante, Préstamo y Biblioteca, favoreciendo la organización del código, la reutilización de componentes y la separación de responsabilidades.

En función de lo expuesto, el presente trabajo tiene como objetivo diseñar un sistema básico de gestión para una biblioteca universitaria mediante los fundamentos de programación en Python y el enfoque de programación orientada a objetos. La propuesta contempla la administración de libros, el registro de estudiantes y el control de los préstamos y las devoluciones, incorporando restricciones que garanticen la coherencia de las operaciones. De esta manera, se busca desarrollar una solución académica funcional que permita aplicar los principios de análisis y diseño de software a una problemática concreta, sentando las bases para futuras mejoras relacionadas con la seguridad, la persistencia de datos y la ampliación de los servicios bibliotecarios.

OBJETIVO GENERAL

Diseñar un sistema básico de gestión para una biblioteca universitaria que permita administrar libros, registrar estudiantes y controlar préstamos y devoluciones mediante los fundamentos de programación en Python y el enfoque de programación orientada a objetos.

- **ANÁLISIS Y DISEÑO DEL SISTEMA**

1. ANÁLISIS DE REQUERIMIENTO

El análisis de requerimientos constituye una etapa fundamental del desarrollo de software, debido a que permite identificar las necesidades de los usuarios, establecer las funcionalidades esperadas y definir las condiciones que debe cumplir el sistema. La norma ISO/IEC/IEEE 29148:2018 establece lineamientos para especificar, documentar y gestionar los requerimientos de sistemas y productos de software, procurando que estén formulados de manera clara y verificable (International Organization for Standardization [ISO], 2018). En concordancia con este enfoque, el sistema propuesto busca organizar los procesos básicos de una biblioteca universitaria mediante operaciones definidas y reglas que permitan controlar los materiales bibliográficos.

1.1. IDENTIFICA Y DESCRIBE 5 REQUERIMIENTOS FUNCIONALES.

Los requerimientos funcionales describen los servicios y las operaciones que el sistema debe proporcionar para satisfacer las necesidades de sus usuarios. Para Sommerville (2021), la especificación de requerimientos forma parte esencial de la ingeniería de software, pues permite establecer las capacidades que debe ofrecer una solución tecnológica. En este proyecto, los requerimientos se identifican mediante códigos para facilitar su documentación, implementación y posterior comprobación.

1.1.1. RF01. Registro de libros

El sistema debe permitir registrar libros mediante un código identificador único, título y autor, almacenando sus datos y asignándoles un estado inicial de disponibilidad. Esta funcionalidad permitirá mantener un catálogo organizado y facilitar la administración de los recursos bibliográficos. Desde la perspectiva de la programación orientada a objetos, la información y las operaciones relacionadas con cada libro pueden representarse mediante una clase con atributos y métodos específicos (Python Software Foundation, s. f.).

1.1.2. RF02. Registro de estudiantes

El sistema debe permitir registrar estudiantes mediante un código de identificación, nombres y carrera profesional, con el propósito de disponer de información suficiente para identificar a los usuarios que solicitan materiales bibliográficos. Este requerimiento contribuirá a establecer una relación verificable entre cada estudiante y las operaciones que realice dentro de la biblioteca.

1.1.3. RF03. Consulta de libros

El sistema debe permitir consultar los libros registrados, visualizar sus datos principales y comprobar si se encuentran disponibles o prestados. Esta funcionalidad facilitará la localización de materiales bibliográficos y permitirá verificar su disponibilidad antes de iniciar una operación de préstamo. Su implementación deberá mantener coherencia entre la información almacenada y el estado real registrado para cada libro.

1.1.4. RF04. Registro de préstamos

El sistema debe permitir registrar el préstamo de un libro asociándolo con un estudiante previamente registrado y una fecha de préstamo válida. La operación solamente podrá ejecutarse cuando el libro se encuentre disponible y los datos requeridos sean correctos. Esta condición responde a la necesidad de definir reglas operativas verificables, en concordancia con los principios de especificación de requerimientos establecidos por la norma ISO/IEC/IEEE 29148:2018 (ISO, 2018).

1.1.5. RF05. Registro de devoluciones

El sistema debe permitir registrar la devolución de un libro previamente prestado y actualizar su estado a disponible, siempre que exista un préstamo vigente asociado a dicho material. Esta funcionalidad permitirá mantener actualizado el catálogo y evitar inconsistencias entre los préstamos registrados y la disponibilidad de los libros.

1.2. LISTE 3 RESTRICCIONES NECESARIAS PARA SU FUNCIONAMIENTO.

Las restricciones establecen condiciones que limitan las operaciones permitidas o determinan cómo debe funcionar una solución informática. La ISO/IEC/IEEE 29148:2018

contempla las restricciones como limitaciones que pueden afectar el sistema, su diseño o su implementación (ISO, 2018). Para la biblioteca universitaria, se proponen las siguientes:

1.2.1. R01. Restricción de disponibilidad de libros

Un libro que se encuentre prestado no podrá asignarse a otro estudiante hasta que se registre su devolución. El sistema deberá comprobar su estado antes de aceptar una nueva operación de préstamo.

Justificación: Esta restricción evita la duplicidad de préstamos incompatibles y contribuye a mantener la consistencia de la información bibliográfica

1.2.2. R02. Restricción de validación de datos

Los registros de libros y estudiantes deberán contener los campos obligatorios y datos válidos. El sistema no deberá procesar préstamos cuando falte información esencial o cuando los identificadores proporcionados no correspondan a registros existentes.

Justificación: La validación permite reducir errores de registro y evita que el sistema realice operaciones sobre información incompleta o inexistente.

1.2.3. R03. Restricción de integridad de los préstamos

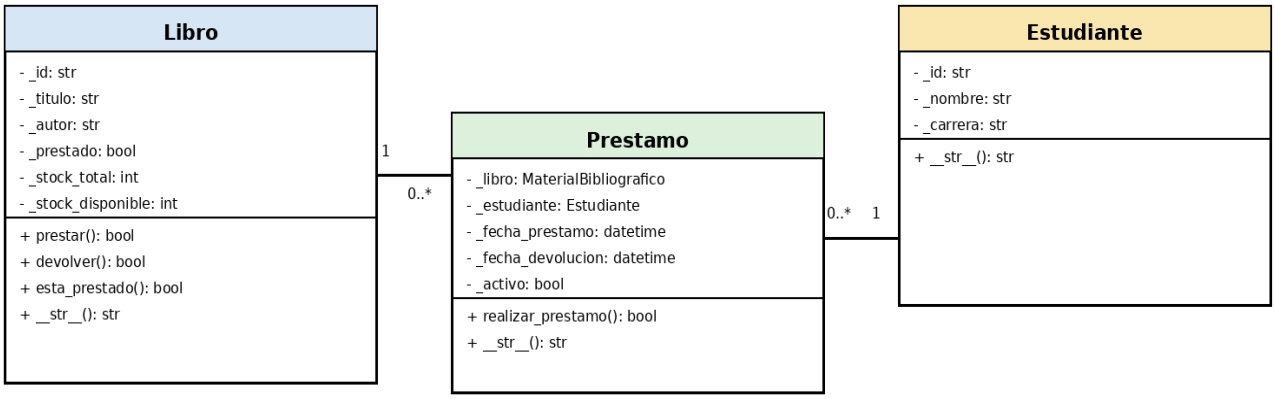
Todo préstamo deberá vincular un libro registrado, un estudiante registrado y una fecha válida. Asimismo, el sistema deberá impedir la devolución de un libro cuando no exista un préstamo vigente que justifique dicha operación.

Justificación: Esta restricción permite conservar la coherencia entre los registros y asegurar que las operaciones de préstamo y devolución sigan una secuencia lógica.

2. **DISEÑO ORIENTADO A OBJETOS:** Diseñe el diagrama de clases que incluya:

Para el diseño partí del diagrama entidad-relación de la biblioteca, que tiene las entidades Autor, Libros, Ejemplar, Usuario y Préstamo, y lo pasé a clases. Cada entidad se convierte en una clase y sus atributos pasan a ser los atributos de la clase. Los atributos empiezan con _ porque son privados (encapsulación).

Diagrama de clases - Parte 1



un material puede aparecer en varios préstamos

cada préstamo corresponde a un estudiante y un material

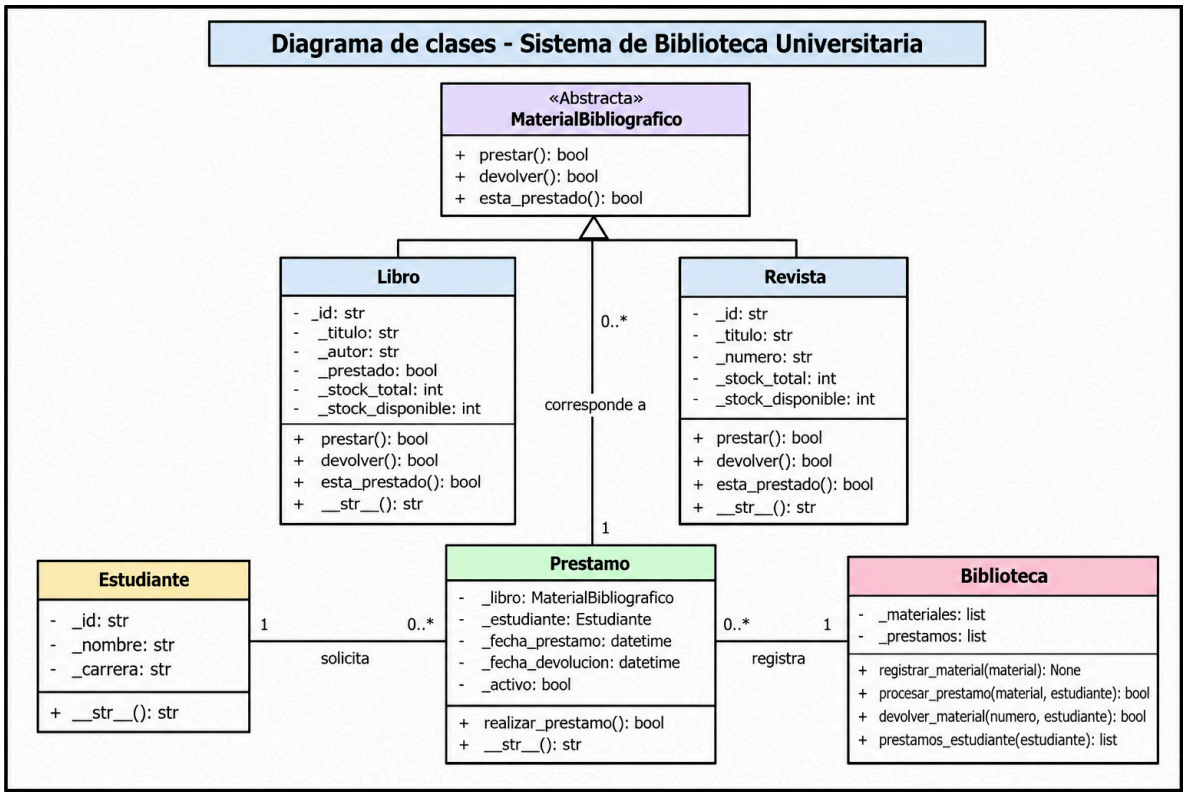


Diagrama entidad-relación	Diagrama de clases
Libros: Código, Título, ISBN, N. páginas, Editorial	Clase Libro: _id, _titulo, _isbn, _num_paginas, _editorial
Autor: Código, Nombre (relación Trabajo con Libros)	Atributo _autor de Libro (se simplifica, ver más abajo)
Usuario: Código, Nombre, Apellido, Dirección, Teléfono	Clase Estudiante: _id, _nombre, _apellido, _direccion, _telefono y _carrera (pedida en la consigna)
Préstamo: Fecha de entrega, Fecha de devolución	Clase Prestamo: _fecha_prestamo, _fecha_devolucion, y las referencias _libro y _estudiante
Ejemplar: Código, Localización	No se crea clase aparte; el atributo _prestado de Libro indica si está disponible

2.1. Clase **Libro** con sus atributos y métodos básicos.

Guarda los datos del libro que vienen del diagrama: _id (el código), _titulo, _isbn,

_num_paginas, _editorial y _autor. Agrego _prestado (True o False) para saber si el libro está disponible. Sus métodos básicos son prestar() y devolver(), que cambian el estado, esta_prestado(), que lo consulta, y __str__ para mostrarlo en pantalla. Se accede a los atributos con properties.

2.2. Clase **Estudiante** que relacione un estudiante con un libro.

Representa al Usuario del diagrama. Guarda los datos de quien pide el libro:

_id, _nombre, _apellido, _direccion y _telefono, más la _carrera que pide la consigna. Solo guarda datos; la tarea de relacionar al estudiante con un libro la hace la clase Prestamo.

2.3. Clase **Préstamo** que relacione un estudiante con un libro.

Relaciona un estudiante con un libro, igual que la entidad Préstamo del diagrama. Guarda _libro, _estudiante, la _fecha_prestamo (la fecha de entrega) y la

_fecha_devolucion. Su método realizar_prestamo() revisa que el libro esté disponible antes de prestarlo. Es una clase aparte porque el préstamo es un hecho con fechas que no pertenece solo al libro ni solo al estudiante.

Relaciones. Cada Prestamo se une a un solo Libro y a un solo Estudiante. Un libro o un estudiante pueden tener muchos préstamos a lo largo del tiempo (0..*).

Decisiones de simplificación. El diagrama tiene dos entidades más. Autor lo dejé como atributo _autor de Libro, porque para este sistema básico basta con el nombre. Ejemplar (cada copia física del libro) no lo modelé como clase para mantener el diseño en el nivel básico que pide el curso: se asume una copia por libro y su estado se guarda en _prestado.

3. Justifique brevemente cada elemento del diseño:

- **Libro:** Representa el material principal solicitado en la consigna y encapsula sus datos mediante atributos privados por convención y propiedades.
- **Estudiante:** Mantiene los datos esenciales de la persona que solicita un material, evitando mezclar esa información con la lógica del préstamo.
- **Prestamo:** Separa el registro de una operación de préstamo de los datos del libro y del estudiante, y conserva la fecha correspondiente.
- **MaterialBibliografico:** Abstrae las operaciones comunes para que distintos materiales puedan tratarse a través de una misma interfaz.
- **Revista:** Permite demostrar herencia y polimorfismo, ya que implementa las operaciones comunes con el comportamiento propio de una revista.
- **Biblioteca:** Centraliza las reglas del catálogo y el control de préstamos y devoluciones, reduciendo la duplicación de validaciones.

1. Alternativas de solución consideradas

• **Estado del libro:** podía guardar el estado como texto ("disponible" o "prestado") o como True/False. Elegí True/False (_prestado) porque es más simple de comparar y evita errores de escritura.

• **Dónde guardar el préstamo:** podía poner el nombre del estudiante y la fecha dentro de Libro, pero eso mezcla datos. Elegí crear la clase Prestamo para que cada clase tenga una sola responsabilidad.

• **Manejo de errores:** podía devolver solo True/False cuando algo falla, pero un error personalizado explica mejor qué pasó. Elegí usar excepciones para el caso de material ya prestado.

2. Fundamentos aplicados

• **Lógica y condicionales:** la regla "un libro prestado no puede prestarse" se resuelve con un if que revisa el estado antes de cambiarlo.

• **Principios de programación orientada a objetos:** encapsulación (atributos privados con _ y properties), herencia y polimorfismo (en la Parte 3, con la clase abstracta MaterialBibliografico).

• **Manejo de fechas:** el módulo datetime permite guardar la fecha del préstamo y calcular la fecha de devolución sumando días con timedelta.

Fuentes

- Cloud-based approach to library management solution for college libraries. (2021). Information Discovery and Delivery, 49(4), 308–318. https://doi.org/10.1108/IDD-10-2019-0076

- Chen, Y. (2023). Research on the application of cloud computing in university library management system. Journal of Education and Educational Research, 5(1), 150–153. https://doi.org/10.54097/jeer.v5i1.11936
- International Organization for Standardization. (2018). ISO/IEC/IEEE 29148:2018: Systems and software engineering—Life cycle processes—Requirements engineering. https://www.iso.org/standard/72089.html
- Python Software Foundation. (s. f.). Classes. Python documentation. https://docs.python.org/3/tutorial/classes.html
- Sommerville, I. (2021). Software engineering (10th ed.). Pearson.